

INDUSTRIAL AI · 2026



# RootIQ

## PLC Error Intelligence — a secure agent harness

Reads a fault log, reuses past fixes, mines the IQAN forum, and synthesises a **cited** solution — behind a security interlock that screens every action.

**Victor Otu Hayford**

 [github.com/novoha/rootiq](https://github.com/novoha/rootiq)

 [rootiq.streamlit.app](https://rootiq.streamlit.app)

## THE PROBLEM

# A cryptic fault — and the fix is buried in a forum

```
# IQAN controller system log
XC44 ; No contact
COUT-Suction hose 1 (C1:43/59) ; Open load
VIN-Joystick (C1:26) ; Low error
MC43FS[0] ; No contact
```

- The answer lives across forum threads & comments — slow to find by hand
- The **same faults recur**, yet nothing is captured or reused
- Sensitive sites want it to run **offline — no cloud, no API key**

**RootIQ** turns that moment into a guided, cited fix — and remembers it for next time.

# Not just an app — a reusable, secure agent harness



## Intake

Upload PDF/image, type a code, or drop a whole CSV export



## Reuse

Instant history cache hit — no LLM, no network



## Retrieve

Offline forum index → scrape threads + comments



## Synthesise

LLM reasons across sources → a cited JSON fix



## Remember

Auto-saved → a knowledge map of related faults



## Guard

security\_check() screens every tool call first



Offline — local Ollama



Online — any OpenAI-compatible API

One sidebar toggle

## WHY IT MATTERS

# A daily bridge between PLC programming and software



### Known faults → instant reuse

The history cache turns a recurring fault into a zero-effort, no-LLM, no-network answer. The same faults come back — now solved on sight. **This saves the most time.**



### New faults → reason + search

Unseen faults are searched against the forum index, scraped with community comments, and reasoned into a cited fix — then **remembered**, so tomorrow they are "known" too.



**The knowledge compounds** — every diagnosis grows the cache and the map, so the tool gets faster and smarter the more it is used.

## HOW IT WORKS

# An orchestrated pipeline — the harness routes, the LLM only synthesises



🔒 `security_check()` runs **before every tool** · every rejection logged to `rejection_log.jsonl`

Known faults short-circuit at **History** — instant, no LLM, no network. Only new faults reach scrape + synthesis.

# Upload a log · type a code · or batch a whole CSV

The screenshot displays the RootIQ web application interface. On the left is a dark sidebar with navigation links: 'app', 'History', 'Reports', 'Settings', and 'Map'. Below these is the 'RootIQ' logo and tagline 'PLC Error Intelligence — Diagnose faster, fix smarter.' Further down, there are status indicators: 'Online mode' (switched on), 'LLM unavailable — solutions from history only', and '2999 topics indexed'. At the bottom of the sidebar are links to 'Pages: Analyse · Map · History · Reports · Settings'.

The main content area is titled 'Analyse PLC Error Log' and has a 'Deploy' button in the top right. It features three tabs: 'Upload a log', 'Type an error', and 'Batch (CSV)'. The 'Batch (CSV)' tab is active. Below the tabs, a text box explains: 'Upload a CSV of error logs, pick the column(s) that describe the fault, and diagnose every unique fault in one click. Known faults resolve instantly from history; new ones are scraped from the forum.' Below this is an 'Upload CSV' section with a file upload area showing '\_sample\_log.csv' (0.7KB). Below the upload area, it states '10 rows · 4 columns' and provides a 'Preview (first 20 rows)' button. A section titled 'Which column(s) describe the error? (combined per row)' shows selected columns 'Description' and 'Value'. Below this, an 'Ignore rows containing any of (comma-separated, case-insensitive)' field contains the text 'System started, Application changed, User logged, Clock changed, Machine ID changed, Date and time not set'. It notes 'Filtered out 1 routine/non-fault row(s)'. A summary states '9 unique faults found across the log.' with a 'See the unique faults' button. A yellow warning box says 'LLM not ready (Online · api.openai.com · gpt-4o). New faults will return "cannot synthesise"; known faults still resolve from history.' At the bottom is a large red button labeled 'Diagnose all faults'.

- A real IQAN system log — **semicolon-delimited, BOM** — parsed cleanly
- De-duplicated to **9 unique faults** across the log
- One click: known faults instant, new ones scraped & reasoned

Routine rows (System started, App changed...) are filtered out automatically.

# Reasoned across multiple threads — never one

## XC44 — No contact

High confidence

**Root cause:** expansion module lost CAN communication — usually termination or wiring [1].

1 · Verify 120  $\Omega$  termination at both ends [1]

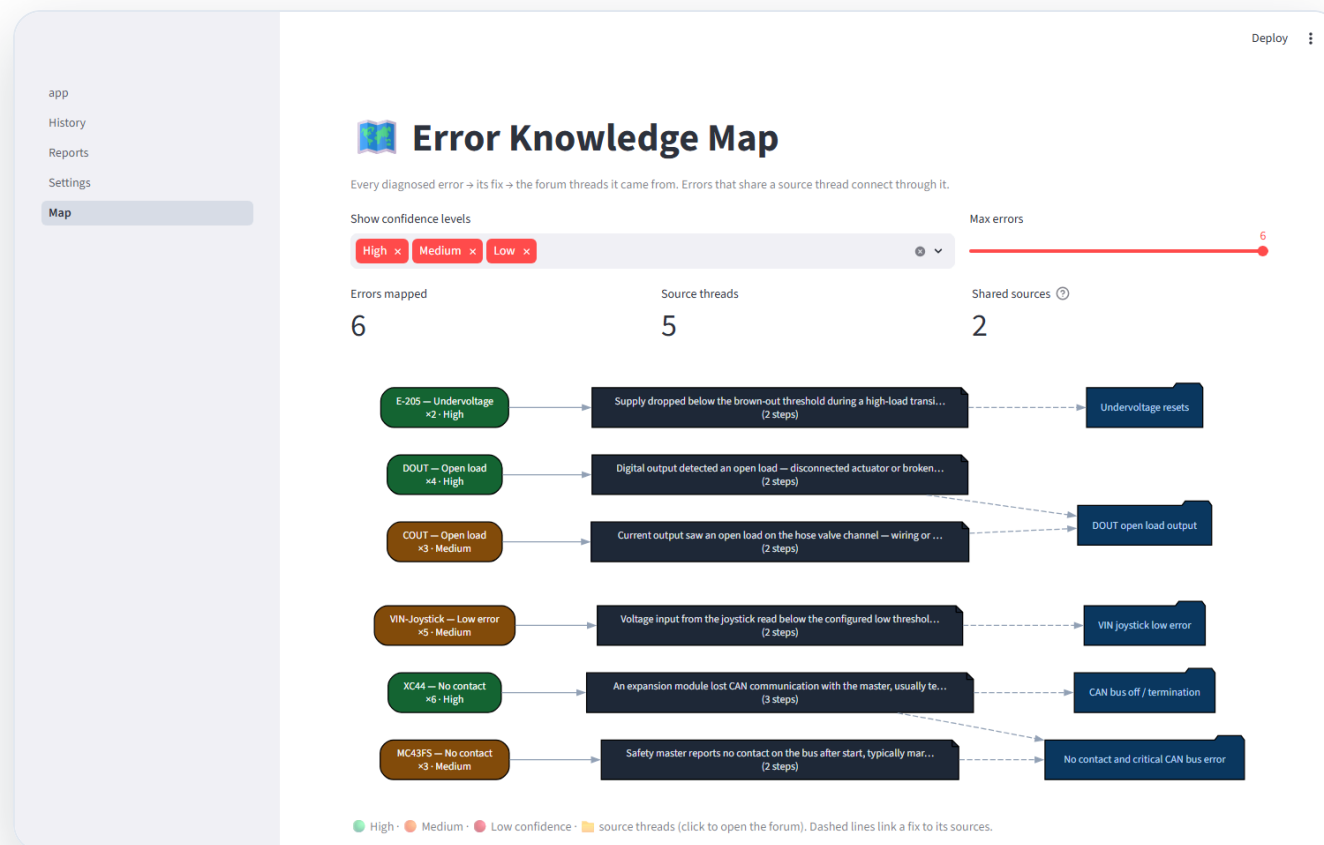
2 · Inspect CAN\_H/CAN\_L for opens/shorts [1][2]

 Sources: No contact & critical CAN error · CAN bus-off / termination

- **Inline citations** [1][2] on every step + all source links
- Weighs **higher-voted, answered** community comments
- Output is parsed **only as JSON** — the model is never wired to a file/command tool

# Related faults connect through shared forum threads

- Each **unique error** is one node, sized by occurrence
- Linked to its **fix** and the **forum threads** it cited
- Shared threads converge** → related faults visibly connect





# Sandboxed tools · drop-in skills · editable config



## Sandboxed tools

read · write · create · list ·  
**transform\_markdown** (md→HTML) ·  
run\_command — every path through  
\_safe\_path().



## Drop-in skills

Any `skills/*.py` with a SKILL dict is auto-  
discovered. **Drop one in — it appears.** No  
harness edit.



## Editable config

`mcp.json` sets the LLM provider, forums,  
and scraping limits — no code change.

The loaded extension: **extract\_log** (problem log → text + codes), plus `lookup_history`, `save_solution`, `phase1_crawl`, `phase2_scrape`.

# A wide attack surface, guarded in four layers

1

## **security\_check(tool, args)**

Regex + command screen before every tool call

2

## **\_safe\_path()**

Authoritative sandbox — resolve() + is\_relative\_to

3

## **check\_url()**

Scheme check + domain allowlist before any HTTP

4

## **rejection\_log.jsonl**

Every block recorded and shown in Settings

```
security_check("run_command", {"command":"rm -rf /"}) → (False, 'Blocked command: rm')  
_safe_path("../../etc/passwd") → raises SandboxError
```

# Each risk: blocked, mitigated, or honestly accepted

| Tool / surface     | Risk                                | Verdict                                                                                       |
|--------------------|-------------------------------------|-----------------------------------------------------------------------------------------------|
| File ops           | path traversal out of the sandbox   |  Blocked   |
| run_command        | shell injection from tool output    |  Mitigated |
| transform_markdown | HTML / script injection             |  Mitigated |
| Outbound HTTP      | SSRF / scraping abuse               |  Mitigated |
| Prompt injection   | scraped / file text as instructions |  By design |
| Secrets            | API key leakage                     |  Mitigated |
| Skills in skills/  | arbitrary import-time code          |  Accepted  |

Honest acceptance of a risk you understand — not pretending it isn't there.

## HOW IT MAPS TO THE ASSIGNMENT

# All four deliverables, in the repo

### ✓ The harness

Working code; app runs headless with zero exceptions

### ✓ A loaded extension

Five drop-in skills — headline extract\_log

### ✓ README + security

Per-tool risks → blocked / mitigated / accepted + diagrams

### ✓ Presentation

This deck + live app screenshots

Offline (Ollama)

Safety interlock + log

Eval / retry loop

Context caps

Staged multi-agent roles

# Stated plainly — the brief rewards this

- **mcp.json is config, not a literal MCP server** — the real extension mechanism is the drop-in skills/ loader
- **Skills are not sandboxed** — trusted first-party code; importing one equals editing the app
- **python is allow-listed** for run\_command (capable) — accepted convenience
- **No robots.txt parsing** — we rate-limit, identify our UA, read only public threads
- **Hosted history is ephemeral** — Streamlit Cloud resets on reboot; persistence needs a database

TRY IT YOURSELF

# Run RootIQ in two ways

```
# Offline – local, no key  
ollama serve && ollama pull phi3  
streamlit run app.py # toggle 🌐 OFF
```

```
# Online – hosted LLM  
export OPENAI_API_KEY=sk- ...  
streamlit run app.py # toggle 🌐 ON
```

 [github.com/novoha/rootiq](https://github.com/novoha/rootiq)

 [rootiq.streamlit.app](https://rootiq.streamlit.app)

# Thank you 🙏

RootIQ — diagnose faster, fix smarter. Securely.

**Victor Otu Hayford** · Industrial AI, 2026

 [github.com/novoha/rootiq](https://github.com/novoha/rootiq)

 [rootiq.streamlit.app](https://rootiq.streamlit.app)